

Antigravity Bridge Extension — Source & Test Log

Files: extension.js, lsClient.js, pickerHandler.js — plus PowerShell test transcripts

A small local HTTP bridge (port 9999) that lets scripts outside the IDE talk to a running Antigravity IDE window — send a prompt into a specific past conversation by title or uuid, and approve or reject the permission prompts it raises, all without touching the mouse. This document is the source for the three files behind it, plus a short transcript proving the send → permission-prompt → accept round trip actually works end to end.

1. extension.js

extension.js

```
const http = require('http');
const vscode = require('vscode');
const { handlePickerRequest } = require('./pickerHandler');
const lsClient = require('./lsClient');

let server = null;

function activate(context) {
  console.log('[Antigravity-Bridge] Extension activated. Starting HTTP server on port 9999...');

  server = http.createServer(async (req, res) => {
    // CORS Headers for Dashboard Web UI
    res.setHeader('Access-Control-Allow-Origin', '*');
    res.setHeader('Access-Control-Allow-Methods', 'POST, GET, OPTIONS');
    res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

    if (req.method === 'OPTIONS') {
      res.writeHead(204);
      res.end();
      return;
    }

    const pathname = (req.url || '/').split('?')[0].replace(/\$/ , '') || '/';

    if (req.method === 'POST' && (pathname === '/send' || pathname === '/prompt')) {
      let body = '';
      req.on('data', chunk => { body += chunk; });
      req.on('end', async () => {
        let data;
        try {
          data = JSON.parse(body || '{}');
        } catch (e) {
          res.writeHead(400, { 'Content-Type': 'application/json' });
          res.end(JSON.stringify({ ok: false, error: 'Invalid JSON body in request: ' + e.message }));
          return;
        }

        try {
          const promptText = data.prompt || data.text || '';
          const targetUuid = data.uuid || data.chat_uuid || data.conversation_id || null;
          const targetIdentifier = data.uuid || data.title || data.conversation || null;

          if (!promptText) {
            res.writeHead(400, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ ok: false, error: 'Empty prompt' }));
            return;
          }

          // 0. Targeted send: identifier (uuid or title) resolves against
          // trajectorySummaries and delivers straight through the backend
          // RPC (SendUserCascadeMessage) - bypasses the UI/picker/focus
          // entirely, so it works regardless of what's currently open.
          if (targetIdentifier) {
            try {
              const match = lsClient.resolveConversation(targetIdentifier);
```

```

        await lsClient.sendUserCascadeMessage(match.cascadeId, promptText);
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({
            ok: true,
            status: 'Prompt sent via direct RPC (SendUserCascadeMessage)',
            cascadeId: match.cascadeId,
            title: match.title
        }));
    } catch (err) {
        res.writeHead(400, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({
            ok: false,
            error: err.message,
            candidates: err.candidates
        }));
    }
    return;
}

// 1. Copy prompt to clipboard safely
try {
    await vscode.env.clipboard.writeText(promptText);
} catch (clipErr) {}

// 2. Open chat view and inject query with target UUID if present
let handled = false;
let lastError = null;

const targets = [];
if (targetUuid) {
    targets.push(
        { cmd: 'antigravity.sendPromptToAgentPanel', arg: { prompt: promptText, conversationId: targetUuid }, session: session },
        { cmd: 'antigravity.sendPromptToAgentPanel', arg: { prompt: promptText, conversationId: targetUuid }, session: session },
        { cmd: 'workbench.action.chat.open', arg: { query: promptText, conversationId: targetUuid, session: session } }
    );
}
targets.push(
    { cmd: 'antigravity.sendPromptToAgentPanel', arg: promptText },
    { cmd: 'workbench.action.chat.open', arg: { query: promptText } },
    { cmd: 'workbench.action.chat.open', arg: promptText }
);

for (const t of targets) {
    try {
        await vscode.commands.executeCommand(t.cmd, t.arg);
        handled = true;
        break;
    } catch (e) {
        lastError = e.message;
    }
}

// 3. If direct injection didn't handle it, open/focus the agent panel safely
if (!handled) {
    const focusTargets = [
        'antigravity.agentSidePanel.focus',
        'antigravity.openAgent',
        'antigravity.toggleChatFocus',
        'antigravity.agentSidePanel.open',
        'workbench.action.chat.open'
    ];

    for (const cmd of focusTargets) {
        try {
            await vscode.commands.executeCommand(cmd);
            handled = true;
            break;
        } catch (e) {}
    }
}

res.writeHead(200, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({

```

```

        ok: true,
        status: 'Prompt received and injected',
        handled,
        target_uuid: targetUuid
    }));

    } catch (err) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: false, error: err.message }));
    }
});

} else if (req.method === 'GET' && pathname === '/conversations') {
    try {
        const list = lsClient.listConversations();
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: true, conversations: list }));
    } catch (e) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: false, error: e.message }));
    }
} else if (req.method === 'GET' && pathname === '/commands') {
    try {
        const allCmds = await vscode.commands.getCommands(true);
        const filtered = allCmds.filter(c =>
            c.toLowerCase().includes('chat') ||
            c.toLowerCase().includes('antigravity') ||
            c.toLowerCase().includes('prompt') ||
            c.toLowerCase().includes('agent') ||
            c.toLowerCase().includes('conversation') ||
            c.toLowerCase().includes('session') ||
            c.toLowerCase().includes('switch') ||
            c.toLowerCase().includes('load') ||
            c.toLowerCase().includes('pick')
        );
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: true, commands: filtered }));
    } catch (e) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: false, error: e.message }));
    }
} else if (req.method === 'POST' && (pathname === '/new_chat' || pathname === '/new')) {
    const newChatCmds = ['antigravity.startNewConversation', 'workbench.action.chat.new', 'antigravity.newChat'];
    for (const cmd of newChatCmds) {
        try {
            await vscode.commands.executeCommand(cmd);
            break;
        } catch (e) {}
    }
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ ok: true, status: 'New conversation started' }));
} else if (req.method === 'POST' && (pathname === '/accept' || pathname === '/approve' || pathname === '/allow')) {
    try {
        await vscode.commands.executeCommand('antigravity.acceptAgentStep');
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: true, status: 'Accepted agent step via antigravity.acceptAgentStep' }));
    } catch (e) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: false, error: e.message }));
    }
} else if (req.method === 'POST' && (pathname === '/reject' || pathname === '/deny')) {
    try {
        await vscode.commands.executeCommand('antigravity.rejectAgentStep');
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: true, status: 'Rejected agent step via antigravity.rejectAgentStep' }));
    } catch (e) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: false, error: e.message }));
    }
} else if (req.method === 'POST' && (pathname === '/picker' || pathname === '/switch')) {
    await handlePickerRequest(req, res);
} else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Not Found' }));
}

```

```

    }
  });

  const PORT = process.env.BRIDGE_PORT ? parseInt(process.env.BRIDGE_PORT, 10) : 9999;

  server.on('error', (err) => {
    console.error('[Antigravity-Bridge] Error:', err.message);
    if (err.code === 'EADDRINUSE') {
      vscode.window.setStatusBarMessage(`■■ Antigravity Bridge Port ${PORT} occupied`, 5000);
    }
  });

  server.listen(PORT, '127.0.0.1', () => {
    console.log(`[Antigravity-Bridge] Server listening on http://127.0.0.1:${PORT}`);
    vscode.window.setStatusBarMessage(`■ Antigravity Bridge Active on port ${PORT}`, 5000);
  });

  let disposable = vscode.commands.registerCommand('antigravityBridge.startServer', () => {
    vscode.window.showInformationMessage('Antigravity Bridge HTTP Server is running on port 9999');
  });

  context.subscriptions.push(disposable);
  context.subscriptions.push({
    dispose: () => {
      if (server) server.close();
    }
  });
}

function deactivate() {
  if (server) {
    server.close();
  }
}

module.exports = {
  activate,
  deactivate
};

```

2. lsClient.js

lsClient.js

```
const fs = require('fs');
const path = require('path');
const os = require('os');
const https = require('https');
const { DatabaseSync } = require('node:sqlite');

const UUID_RE = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
const LOGS_DIR = path.join(os.homedir(), 'AppData', 'Roaming', 'Antigravity IDE', 'logs');
const STATE_DB = path.join(os.homedir(), 'AppData', 'Roaming', 'Antigravity IDE', 'User', 'globalStorage', 'state.vscdb');
const SERVICE = 'exa.language_server_pb.LanguageServerService';

function getConnectionInfo() {
  const dirs = fs.readdirSync(LOGS_DIR)
    .map(name => ({ name, mtime: fs.statSync(path.join(LOGS_DIR, name)).mtimeMs }))
    .sort((a, b) => b.mtime - a.mtime);

  for (const { name } of dirs) {
    const logPath = path.join(LOGS_DIR, name, 'ls-main.log');
    if (!fs.existsSync(logPath)) continue;
    const text = fs.readFileSync(logPath, 'utf8');
    const tokenMatch = text.match(/--csrf_token (\S+)/);
    const portMatch = text.match(/listening on random port at (\d+) for HTTPS/);
    if (tokenMatch && portMatch) {
      return { token: tokenMatch[1], port: parseInt(portMatch[1], 10) };
    }
  }
  throw new Error('Could not find language server csrf token / port in any ls-main.log');
}

function rpcCall(method, body) {
  const { token, port } = getConnectionInfo();
  const payload = JSON.stringify(body);
  return new Promise((resolve, reject) => {
    const req = https.request({
      host: '127.0.0.1',
      port,
      path: `/${SERVICE}/${method}`,
      method: 'POST',
      rejectUnauthorized: false,
      headers: {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(payload),
        'x-codeium-csrf-token': token
      }
    }, res => {
      let data = '';
      res.on('data', chunk => { data += chunk; });
      res.on('end', () => {
        let parsed;
        try { parsed = JSON.parse(data || '{}'); } catch (e) { parsed = { raw: data }; }
        if (res.statusCode !== 200) {
          reject(new Error(`${method} -> HTTP ${res.statusCode}: ${parsed.message || data}`));
          return;
        }
        resolve(parsed);
      });
    });
    req.on('error', reject);
    req.write(payload);
    req.end();
  });
}

// --- Schema-less protobuf reader -----
// Only returns raw {field, wire, value} tuples (value is a Buffer for
// wire type 2) - no auto-interpretation, since only three fixed field
// paths are needed here (see trajectorySummaries structure below).

function readVarint(buf, pos) {
```

```

    let result = 0, shift = 1, byte;
    do {
        if (pos >= buf.length) throw new Error('truncated varint');
        byte = buf[pos++];
        result += (byte & 0x7f) * shift;
        shift *= 128;
    } while (byte & 0x80);
    return [result, pos];
}

function decodeMessage(buf) {
    const fields = [];
    let pos = 0;
    const n = buf.length;
    try {
        while (pos < n) {
            let tag;
            [tag, pos] = readVarint(buf, pos);
            const fieldNo = tag >>> 3;
            const wire = tag & 7;
            if (fieldNo === 0) return null;
            if (wire === 0) {
                let val; [val, pos] = readVarint(buf, pos);
                fields.push({ field: fieldNo, wire: 'varint', value: val });
            } else if (wire === 1) {
                if (pos + 8 > n) return null;
                fields.push({ field: fieldNo, wire: 'fixed64', value: buf.subarray(pos, pos + 8) });
                pos += 8;
            } else if (wire === 5) {
                if (pos + 4 > n) return null;
                fields.push({ field: fieldNo, wire: 'fixed32', value: buf.subarray(pos, pos + 4) });
                pos += 4;
            } else if (wire === 2) {
                let length; [length, pos] = readVarint(buf, pos);
                if (length < 0 || pos + length > n) return null;
                fields.push({ field: fieldNo, wire: 'bytes', value: buf.subarray(pos, pos + length) });
                pos += length;
            } else {
                return null;
            }
        }
    } catch (e) {
        return null;
    }
    return fields;
}

function findField(fields, fieldNo) {
    return fields ? fields.find(f => f.field === fieldNo) : undefined;
}

// --- trajectorySummaries -----
// antigravityUnifiedStateSync.trajectorySummaries in state.vscdb:
//   repeated field 1 = one entry per conversation
//   field 1 = uuid (utf8 string bytes)
//   field 2 = sub-message:
//   field 1 = ANOTHER base64-encoded protobuf (double-wrapped - the
//   raw bytes here are the ASCII characters of a base64
//   string, not the decoded bytes themselves)
//   field 1 = the real title (utf8 string bytes)

function listConversations() {
    const db = new DatabaseSync(STATE_DB, { readOnly: true });
    let raw;
    try {
        const row = db.prepare(
            "SELECT value FROM ItemTable WHERE key = 'antigravityUnifiedStateSync.trajectorySummaries'"
        ).get();
        raw = row && row.value;
    } finally {
        db.close();
    }
    if (!raw) return [];
}

```

```

const top = decodeMessage(Buffer.from(raw, 'base64'));
if (!top) return [];

const results = [];
for (const entry of top) {
  if (entry.field !== 1 || entry.wire !== 'bytes') continue;
  try {
    const entryFields = decodeMessage(entry.value);
    const uuidField = findField(entryFields, 1);
    const subField = findField(entryFields, 2);
    if (!uuidField || !subField) continue;

    const cascadeId = uuidField.value.toString('utf8');
    const subFields = decodeMessage(subField.value);
    const b64Field = findField(subFields, 1);
    if (!b64Field) continue;

    const innerBytes = Buffer.from(b64Field.value.toString('utf8'), 'base64');
    const innerFields = decodeMessage(innerBytes);
    const titleField = findField(innerFields, 1);
    if (!titleField) continue;

    results.push({ cascadeId, title: titleField.value.toString('utf8') });
  } catch (e) {
    // one malformed entry shouldn't break the whole list
  }
}
return results;
}

function resolveConversation(identifier) {
  const conversations = listConversations();

  if (UUID_RE.test(identifier)) {
    const match = conversations.find(c => c.cascadeId.toLowerCase() === identifier.toLowerCase());
    if (!match) {
      throw new Error(`No known conversation with uuid ${identifier}`);
    }
    return match;
  }

  const needle = identifier.toLowerCase();
  const matches = conversations.filter(c => c.title.toLowerCase().includes(needle));

  if (matches.length === 0) {
    throw new Error(`No conversation title matches "${identifier}"`);
  }
  if (matches.length > 1) {
    const err = new Error(`${identifier} matches ${matches.length} conversations, be more specific`);
    err.candidates = matches.map(m => ({ cascadeId: m.cascadeId, title: m.title }));
    throw err;
  }
  return matches[0];
}

// Real shape captured straight off the wire (TLS-decrypted via SSLKEYLOGFILE +
// Wireshark, decoded from a genuine SendUserCascadeMessage call the real UI
// made). The field is `cascadeConfig`, NOT `userConfig` - that one wrong key
// name is exactly why earlier attempts got a silent 200 {} but the agent
// still errored downstream (the config was never actually applied).
const DEFAULT_CASCADE_CONFIG = {
  plannerConfig: {
    conversational: { plannerMode: 'CONVERSATIONAL_PLANNER_MODE_DEFAULT', agenticMode: true },
    toolConfig: {
      runCommand: { autoCommandConfig: { autoExecutionPolicy: 'CASCADE_COMMANDS_AUTO_EXECUTION_OFF' } },
      notifyUser: { artifactReviewMode: 'ARTIFACT_REVIEW_MODE_ALWAYS' },
      permissionConfig: { defaultGrants: { ask: ['read_url(*)'] } }
    },
  },
  requestedModel: { model: 'MODEL_PLACEHOLDER_M71' },
  ephemeralMessagesConfig: { enabled: true },
  knowledgeConfig: { enabled: true },
},
conversationHistoryConfig: { enabled: true }

```

```

};

async function sendUserCascadeMessage(cascadeId, text) {
  return rpcCall('SendUserCascadeMessage', {
    cascadeId,
    items: [{ text }],
    cascadeConfig: DEFAULT_CASCADE_CONFIG
  });
}

module.exports = {
  listConversations,
  resolveConversation,
  sendUserCascadeMessage,
  rpcCall,
  getConnectionInfo
};

if (require.main === module) {
  const http = require('http');
  const PORT = parseInt(process.env.PORT || '9999', 10);

  const server = http.createServer(async (req, res) => {
    res.setHeader('Access-Control-Allow-Origin', '*');
    res.setHeader('Access-Control-Allow-Methods', 'POST, GET, OPTIONS');
    res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

    if (req.method === 'OPTIONS') {
      res.writeHead(204);
      res.end();
      return;
    }

    const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`);
    const pathname = url.pathname.replace(/\/$/, '') || '/';

    if (req.method === 'GET' && pathname === '/conversations') {
      try {
        const list = listConversations();
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: true, conversations: list }));
      } catch (err) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ ok: false, error: err.message }));
      }
      return;
    }

    if (req.method === 'POST' && (pathname === '/send' || pathname === '/prompt')) {
      let body = '';
      req.on('data', chunk => { body += chunk; });
      req.on('end', async () => {
        try {
          const data = JSON.parse(body || '{}');
          const promptText = data.prompt || data.text || '';
          const targetIdentifier = data.uuid || data.title || data.conversation || null;

          if (!promptText) {
            res.writeHead(400, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ ok: false, error: 'Empty prompt' }));
            return;
          }

          let match;
          if (targetIdentifier) {
            match = resolveConversation(targetIdentifier);
          } else {
            const convs = listConversations();
            if (convs.length === 0) throw new Error('No active conversations found in state DB');
            match = convs[0];
          }

          await sendUserCascadeMessage(match.cascadeId, promptText);
        } catch (err) {
          // Error handling for the main logic
        }
      });
    }
  });

  server.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
}

```



```

        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({
            ok: true,
            status: 'Prompt sent via direct RPC (SendUserCascadeMessage)',
            cascadeId: match.cascadeId,
            title: match.title
        }));
    } catch (err) {
        res.writeHead(500, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({
            ok: false,
            error: err.message,
            candidates: err.candidates
        }));
    }
    });
    return;
}

if (req.method === 'POST' && pathname === '/rpc') {
    let body = '';
    req.on('data', chunk => { body += chunk; });
    req.on('end', async () => {
        try {
            const data = JSON.parse(body || '{}');
            const method = data.method;
            const payload = data.payload || {};

            if (!method) {
                res.writeHead(400, { 'Content-Type': 'application/json' });
                res.end(JSON.stringify({ ok: false, error: 'Missing RPC method' }));
                return;
            }

            const result = await rpcCall(method, payload);
            res.writeHead(200, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ ok: true, result }));
        } catch (err) {
            res.writeHead(500, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ ok: false, error: err.message }));
        }
    });
    return;
}

res.writeHead(404, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({ ok: false, error: 'Endpoint not found' }));
});

try {
    const info = getConnectionInfo();
    console.log(`[lsClient] Detected Antigravity IDE RPC Target: port ${info.port}`);
} catch (e) {
    console.warn(`[lsClient] Warning: ${e.message}`);
}

server.listen(PORT, '127.0.0.1', () => {
    console.log(`[lsClient] Standalone RPC HTTP Server running at http://127.0.0.1:${PORT}`);
});
}

```

3. pickerHandler.js

pickerHandler.js

```
const vscode = require('vscode');

/**
 * Opens the past conversations picker in Antigravity IDE.
 */
async function handlePickerRequest(req, res) {
  try {
    let body = '';
    if (req && typeof req.on === 'function') {
      await new Promise((resolve) => {
        req.on('data', chunk => { body += chunk; });
        req.on('end', resolve);
      });
    }
    let data = {};
    try {
      if (body) data = JSON.parse(body);
    } catch (e) {}

    const targets = [
      'antigravity.openConversationPicker',
      'conversationPicker.showConversationPicker',
      'antigravity.openConversationWorkspaceQuickPick',
      'workbench.view.chat.sessions',
      'workbench.action.chat.toggleChatViewSessions'
    ];

    let handled = false;
    let executedCmd = null;

    for (const cmd of targets) {
      try {
        await vscode.commands.executeCommand(cmd);
        handled = true;
        executedCmd = cmd;
        break;
      } catch (e) {}
    }

    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ ok: handled, command: executedCmd, query: data.query || null }));
  } catch (err) {
    res.writeHead(500, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ ok: false, error: err.message }));
  }
}

module.exports = { handlePickerRequest };
```

4. PowerShell Test Transcripts

Test 1 — /send triggers a permission prompt, then /accept approves it

```
PS D:\my_projects\idktool> $body = @{ prompt = 'hi run python -c print("heloo this will ask permission")' } |  
>> Invoke-RestMethod -Uri "http://127.0.0.1:9999/send" -Method POST -ContentType "application/json" -Body $bo
```

```
    ok status                                handled target_uuid  
    -- -----                                -----  
True Prompt received and injected          True
```

```
PS D:\my_projects\idktool> Invoke-RestMethod -Uri "http://127.0.0.1:9999/accept" -Method POST
```

```
    ok status  
    -- -----  
True Accepted agent step via antigravity.acceptAgentStep
```

```
PS D:\my_projects\idktool>
```

Antigravity IDE side: the same prompt raised a real "Allow running this command?" permission card (options: Yes, allow this time / No, tell the agent what to do instead). The Antigravity chat agent itself confirmed: "Your automated /accept bridge request successfully approved and executed the step" — command output returned heloo this will ask permission.

Test 2 — repeat /send, then /accept called twice in a row

```
PS D:\my_projects\idktool> $body = @{ prompt = 'hi run python -c print("heloo this will ask permission")' } |  
>> Invoke-RestMethod -Uri "http://127.0.0.1:9999/send" -Method POST -ContentType "application/json" -Body $bo  
>>
```

```
    ok status                                handled target_uuid  
    -- -----                                -----  
True Prompt received and injected          True
```

```
PS D:\my_projects\idktool> Invoke-RestMethod -Uri "http://127.0.0.1:9999/accept" -Method POST
```

```
    ok status  
    -- -----  
True Accepted agent step via antigravity.acceptAgentStep
```

```
PS D:\my_projects\idktool> Invoke-RestMethod -Uri "http://127.0.0.1:9999/accept" -Method POST
```

```
    ok status  
    -- -----  
True Accepted agent step via antigravity.acceptAgentStep
```

```
PS D:\my_projects\idktool>
```

Endpoint aliases confirmed available for approvals: /accept, /approve, /allow all map to antigravity.acceptAgentStep.

5. Screenshots

[Placeholder — two screenshots not embedded yet. Provide the image files (save/export from the chat) and this section will be filled in with the actual images.]